



Projet 1 LINFO1341 - SRTP

Lai Loïc, Zhang Chi

Architecture Générale

3 fichiers principaux

packet.py

Contient une classe Packet, les objets de cette classe contiennent les informations des packets (Type, window, timestamp, payload).

Cette classe a deux méthodes qui permet de créer un packet et de déballer un packet:

```
def pack(self):  
    """  
    Créer un packet STPR  
    :return: Packet STPR en bytes  
    """  
  
@staticmethod  
def unpack(packet):  
    """  
    Unpack un packet STPR  
    :param data: Les bytes du packet  
    :return: Un objet Packet ou None si invalide  
    """
```

server.py

Contient le code pour lancer le serveur host qui reçoit les packets du client et renvoie au client les données de leur requête, si ce dernier est valide, sous forme de packet.

client.py

Contient le code pour lancer le client du serveur host. Le client envoie des requêtes au serveur (Transfert des données d'un fichier) et reçoit une réponse si la requête est valide.

Gestion du timestamp

Dans timestamp on a mis la valeur que donne `time.time()`.

Utilité:

- Il permet de tracker le temps de création des packets. Il sera utile lorsqu'on veut savoir si un paquet ou un ACK met trop de temps à se transmettre et définir une limite de temps pour les paquet qui ne vont jamais arriver (timeout).
- Il permet d'identifier les paquets qui sont trop vieux en cas de forte congestion
- Il permet d'estimer le temps de voyage d'un packets entre le serveur et le client. Le serveur calcule la différence de temps pour ajuster son comportement.

```
# Découpage du contenu du fichier en packet de taille 1024 max
while True:
    chunk = f.read(1024)
    packets.append(Packet( type: 1, window, seqnum, int(time.time()), chunk))

    if not chunk:
        packet = Packet( type: 1, window, seqnum, int(time.time()), payload=b'')
        sock.sendto(packet.pack(), client_address)
        break
```

Valeur du retransmission timeout

On a choisi le temps du timeout à 0,5 seconde.

Le choix de cette valeur n'a pas été ultra réfléchi, on a choisi cette valeur par simplicité car il fonctionne bien dans les tests simple que nous avons réalisés.

Nous avons bien conscience que une valeur trop petite peu générer des doublons inutiles et une valeur trop grande peut faire attendre le transmission trop longtemps pour rien.

C'est pour cela que nous risquons d'ajuster ce temps durant la semaine à venir suite à la réalisation de tests plus poussé et complexe afin de trouver la valeur optimale.

```
# Retransmission du message due au timeout
now = time.time()
for i in range(last_ack + 1, last_sent + 1):
    if i in timestamp and (now - timestamp[i] > 0.5):
        print(f"Timeout, retransmission du seq {packets[i].seqnum}", file=sys.
              sock.sendto(packets[i].pack(), client_address)
              timestamp[i] = now
```

Partie critique

Actuellement, l'élément qui affecte le plus la vitesse de transfert dans notre code est timeout du socket de 0,1sec lorsque on fait `sock.settimeout(0.1)`

Le serveur va s'arrêter pendant 0,1 sec pour recevoir un ACK avant de vérifier si il doit retransmettre. Cette attente va donc limiter le nombre de paquets envoyés.

On essayera de trouver une solution à ce problème durant la semaine prochaine avant la deuxième remise.

Performance

Nous n'avons malheureusement pas eu assez de temps pour réalisé des tests très poussé et tester les performances du programmes dans des situations complexes.

Néanmoins, le programme fonctionne très bien sur des tests effectués en local et il passe les tests basique de Inginiuous.

Nous allons effectuer des tests plus poussé et donner des résultats concret pour la deuxième remise.

Tests

```
===== tests coverage =====
----- coverage: platform win32, python 3.12.10-final-0 -----

Name                Stmts  Miss  Cover
-----
src\client.py        59      5   92%
src\packet.py        92     14   85%
src\server.py       101     20   80%
test_client.py       58      0  100%
test_packet.py       75      0  100%
test_server.py       87      6   93%
-----
TOTAL                472     45   90%
```